VLSI Internship – Task 3 Report

Introduction to Sequential Circuits, Flip-Flops, Registers and Counters using Verilog HDL

Submitted by

Name	**:**	Vishesh Tyagi
College	**:**	Raj Kumar Goel Institute of Technology
Branch	**:**	Electronics and Computer Engineering
Internship	**:**	VLSI Internship
Task	**:**	Task 3

Table of Contents

1. Introduction

Sequential circuits are an important part of digital system design because they are capable of storing and processing information over time. These circuits generally operate using a clock signal that controls when the stored information is updated.
Flip-flops are the basic memory elements used in sequential digital circuits and can store one bit of information. Flip-flops can also be used to implement counters that change their state in response to clock pulses.
In this task, sequential circuits are designed and implemented using Verilog HDL. The designs include D flip-flop, JK flip-flop, a 4-bit register, and a 4-bit counter.

2. Objective

The objectives of this task are:

- To understand the fundamentals of sequential logic circuits.
- To design flip-flops using Verilog HDL.
- To implement registers and counters using Verilog HDL.
- To develop testbenches for sequential circuits.
- To verify the expected behavior of the RTL designs.To analyze waveform outputs.

3. Software Used

- Xilinx Vivado
- Verilog HDL
- Waveform Viewer

4. Sequential Logic Fundamentals

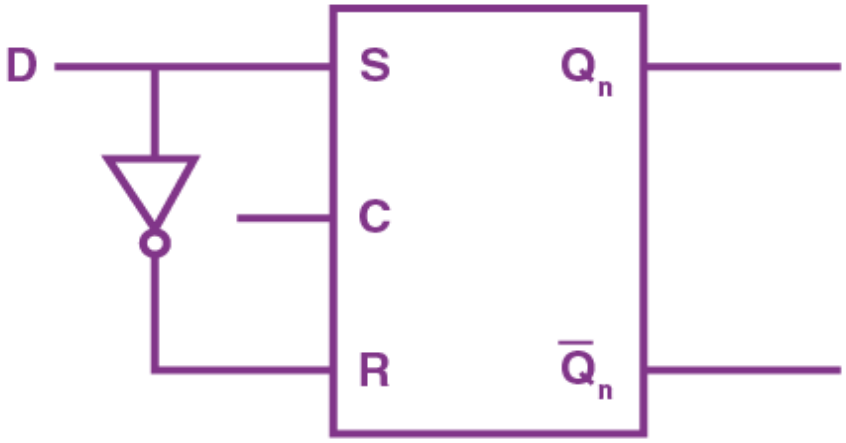
Sequential circuits are digital circuits whose output depends on the current input as well as the previous state. They use memory elements and generally operate with a clock signal.

- **Combinational Logic**: Output depends only on the current input.
- **Sequential Logic**: Output depends on current input and previous state.
- **Clock**: Controls when the circuit updates its state.
- **Flip-Flop**: A basic memory element used to store one bit of data.
- **Latch**: Level-sensitive memory element.
- **Flip-Flop**: Edge-sensitive memory element.
- **Synchronous Circuit**: A sequential circuit whose state changes according to a clock signal.

5 Flip-Flops

5.1 :- D Flip-Flop

A D Flip-Flop is a sequential circuit that stores one bit of data. The input **D** is transferred to output **Q** at the active edge of the clock.



Characteristic Equation :-

- $Q_{next} = D$

Truth Table

Clock Edge	D	Q (Next State)
↑	0	0
↑	1	1
No clock edge	X	Previous Q

Design Code

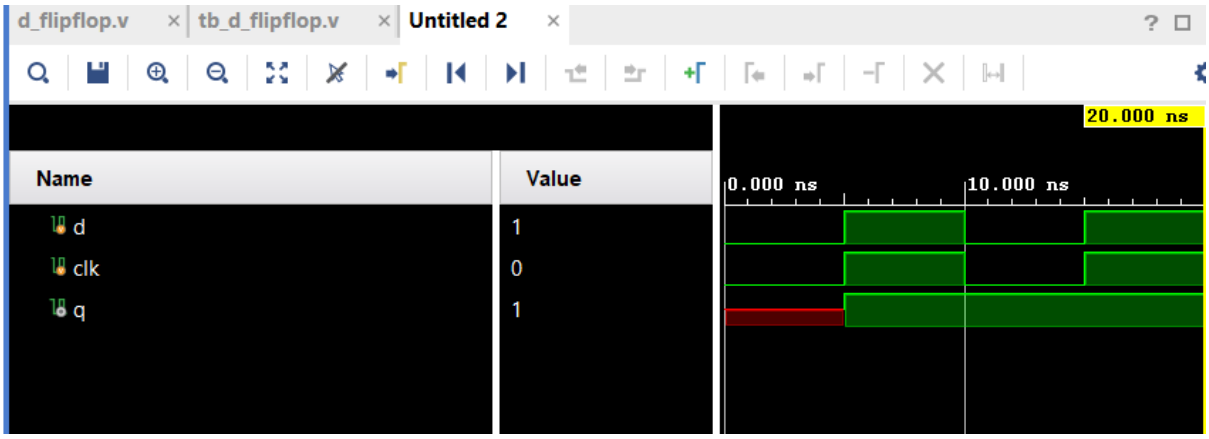
```
module d_flipflop(
input d, clk,
output reg q);
always@(posedege clk)
begin
q <= d;
end
endmodule
```

Testbench

```
module tb_d_flipflop();
reg d, clk;
wire q;
d_flipflop dut(
.d(d),
.clk(clk),
.q(q) );
initial begin
clk = 0;
forever #5 clk = ~clk;
end
initial begin
d=0;#5;
d=1;#5;
d=0;#5;
```

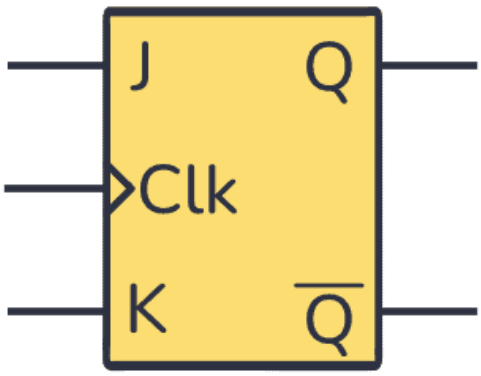
```
$finish; #10
end
endmodule
```

Simulation Waveform



5.2 :- JK Flip-Flop

A JK Flip-Flop is a sequential circuit used to store one bit of data. It has two inputs, J and K, and a clock input.



Characteristic Equation :-

- $Q_{next} = J\overline{Q} + \overline{K}Q$

Truth Table :-

J	K	Q(next)	Operation
0	0	Q	0
0	0	0	1
0	1	1	1
0	1	$\bar{Q}$	0

Design Code

```
module jk_flipflop(
input J,
input K,
input clk,
input reset,
output reg Q );
always @(posedge clk or posedge reset) begin
  if (reset)
Q <= 1'b0;
 else begin
    case ({J,K})
    2'b00: Q <= Q;
    2'b01: Q <= 1'b0;
    2'b10: Q <= 1'b1;
    2'b11: Q <= ~Q;
    endcase
```

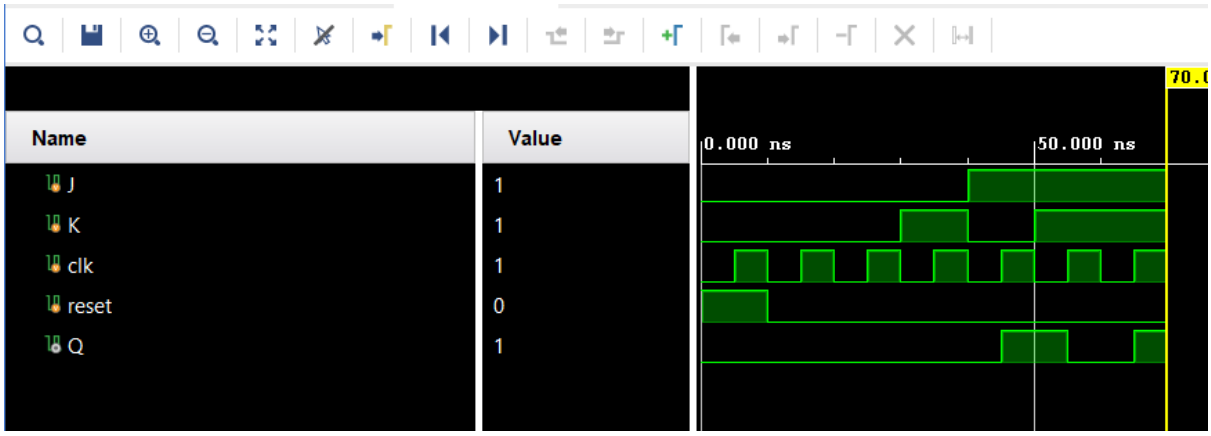
```
  end
end
endmodule
```

Testbench

```
module tb_jk_flipflop();
reg J, K, clk, reset;
wire Q;
jk_flipflop dut (
    .J(J),
    .K(K),
    .clk(clk),
    .reset(reset),
    .Q(Q) );
always #5 clk = ~clk;
initial begin
clk = 0;
    reset = 1;
    J = 0;
    K = 0;
#10 reset = 0;
#10 J = 0; K = 0;   // No change
#10 J = 0; K = 1;   // Reset
#10 J = 1; K = 0;   // Set
#10 J = 1; K = 1;   // Toggle
#10 $finish;
end
```

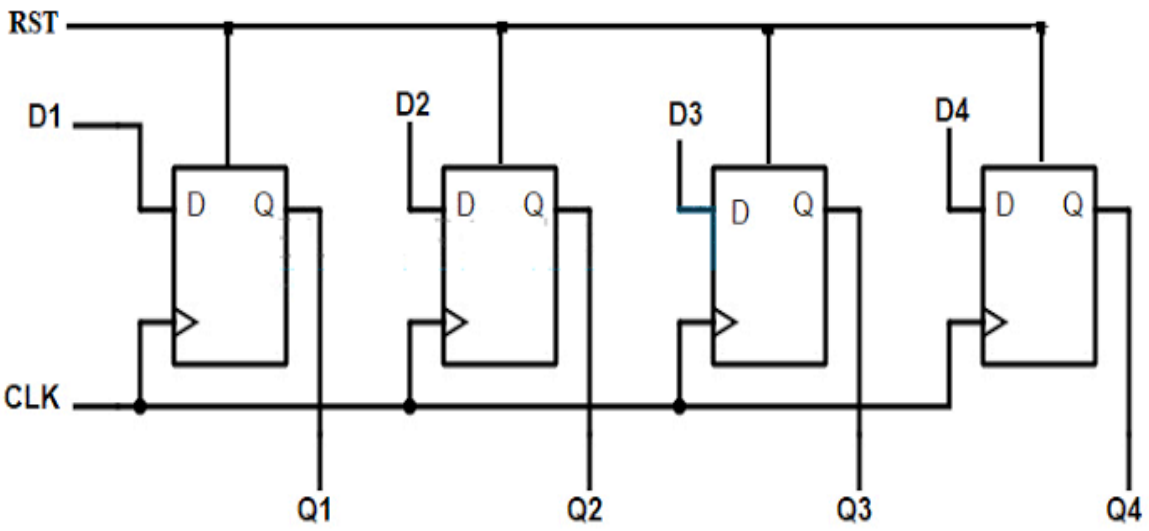
endmodule

Simulation Waveform



6. 4-bit Register

A 4-bit register is a sequential digital circuit that stores 4 bits of binary data. It is made using four flip-flops and typically stores or updates the data on a clock edge.



Truth Table

Enable (EN)	D_3 D_2 D_1 D_0	Q_3 Q_2 Q_1 Q_0	Operation
0	XXXX	Previous value	Hold
1	0000	0000	Load
1	0101	0101	Load
1	1010	1010	Load
1	1111	1111	Load

Design Code

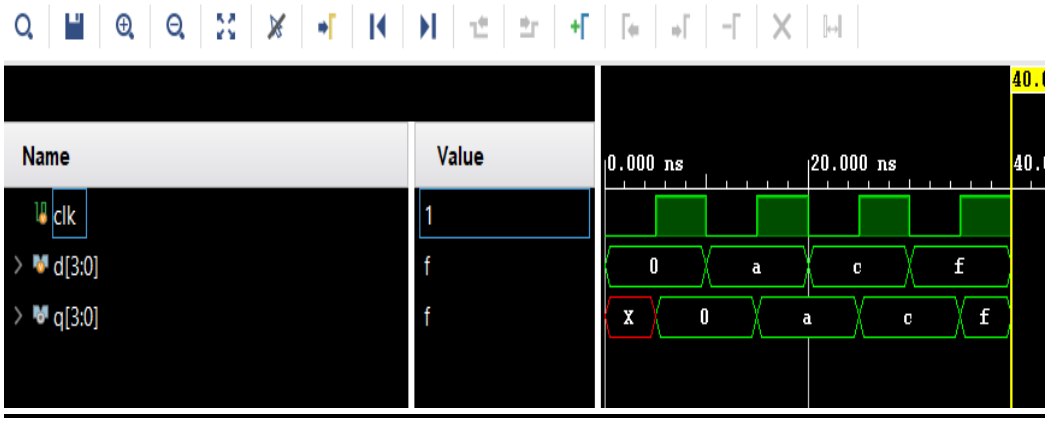
```
module register_4bit(
   input clk,
   input [3:0] d,
   output reg [3:0] q );
always @(posedge clk)
begin
   q <= d;
end
endmodule
```

Testbench

```
module tb_register_4bit();
reg clk;
reg [3:0] d;
wire [3:0] q;
register_4bit dut (
```

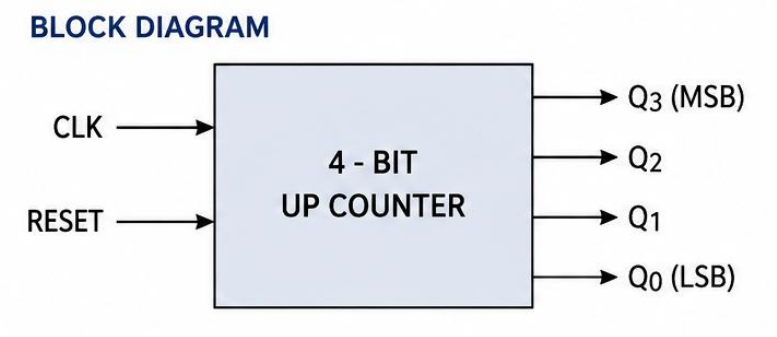
```
    .clk(clk),
    .d(d),
    .q(q) );
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
initial begin
d = 4'b0000; #10;
d = 4'b1010; #10;
d = 4'b1100; #10;
d = 4'b1111; #10;
$monitor("Time=%0t | CLK=%b | D=%b | Q=%b",
          $time, clk, d, q);
$finish;
 end
endmodule
```

Simulation Waveform



7. 4-bit Counter

A 4-bit counter is a sequential digital circuit that stores and counts 4-bit binary values. It changes its state with each active clock edge and can count from 0000 to 1111 (0 to 15) before repeating.



Design Code

```
module counter_4bit(
input clk, reset,
output reg [3:0] q );

always@(posedge clk)
begin
if(reset)
q <= 4'b0000;
else q <= q + 1'b1;
end
endmodule
```

Testbench

```
module tb_counter_4bit( );
reg clk, reset;
wire [3:0]q;
counter_4bit dut(
.clk(clk),
.reset(reset),
.q(q) );
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
initial begin
  reset = 1; #10;
  reset = 0; #10;
  reset = 1; #10;
  $monitor ( "clk:%b,reset:%b,q:%b",clk,q,reset);
  $finish; #10;
end
endmodule
```

8. Results

The four sequential circuits were successfully designed and simulated using Verilog HDL with behavioral modeling. The simulation waveforms were analyzed to verify the expected operation of each circuit.

D Flip-Flop :-

The D flip-flop successfully stored the input data on the active clock edge. The output changed according to the applied D input.

JK Flip-Flop :-

The JK flip-flop was successfully implemented and verified. It performed hold, reset, set, and toggle operations according to the applied J and K inputs.

4-Bit Register :-

The 4-bit register successfully stored the 4-bit input data on every active clock edge. The output followed the input after the clock transition.

4-Bit Counter :-

The 4-bit counter successfully counted from 0000 to 1111 and then returned to 0000, confirming MOD-16 counting operation.

9. Conclusion

The task successfully demonstrated the design and verification of basic sequential circuits using Verilog HDL behavioral modeling. D flip-flop, JK flip-flop, 4-bit register, and 4-bit counter were implemented and simulated successfully.
The corresponding testbenches were used to apply different input conditions, and the simulation waveforms confirmed that the designs behaved as expected. This task provided practical understanding of clocked sequential logic, data storage, flip-flop operation, registers, and counters.

10. Learning Outcomes

- After completing this task, I learned:
- Understood the fundamentals of sequential logic circuits.
- Learned the working of D and JK flip-flops.
- Learned to design a 4-bit register and 4-bit counter.
- Gained practical experience in behavioral modeling using Verilog HDL.
- Learned to create testbenches for sequential circuits